

个人开发日志

基本信息

项目	内容
姓名	[填写]
学号	[填写]
小组编号	[填写]
项目题目	算法过程可视化系统（AlgoVisual）
担任角色	组长 / 项目架构 & 核心框架负责人
负责模块	Electron 桌面端脚手架、TypeScript 类型系统、Zustand 全局状态机、项目配置与打包

一、负责模块与任务

编号	任务	核心产出
T01	搭建项目工程环境	package.json / tsconfig.json / vite.config.ts / tailwind.config.mjs / postcss.config.mjs / index.html
T02	设计核心类型系统	src/types/algo.ts（16种ActionType、8种VisualizerType、20+接口）
T03	实现全局状态机	src/context/AlgoContext.tsx（Zustand Store、14个Action方法）
T04	Electron 桌面壳	electron/main.ts（窗口管理、M1适配、安全策略）
T05	入口文件与全局样式	src/main.tsx / App.tsx / index.css
T06	数据管理子系统	随机生成按算法定制、字符直输、保存/加载、图边输入
T07	打包与发布	electron-builder 配置、Mac ARM64打包、版本管理

二、每日开发日志

2026年6月18日（星期三）

完成内容：

- 1. 初始化项目：npm init，配置 package.json，安装 React 18 / TypeScript 5 / Vite 5 / Tailwind CSS 3 / Zustand 4 / Framer Motion 11
- 2. 配置 tsconfig.json（strict模式、路径别名@/）、tsconfig.node.json（Electron主进程编译）、vite.config.ts（路径解析、端口5173）

3. 配置 tailwind.config.mjs (IDE暗色主题色板: ide-bg/ide-sidebar/ide-panel/ide-border/ide-text/canvas-bg, 自定义keyframes: pulseHighlight/swapJump/fadeIn)
4. 配置 postcss.config.mjs (tailwindcss + autoprefixer)
5. 编写 index.html (JetBrains Mono + Inter 字体CDN、中文lang、暗色body)
6. 编写 src/index.css (Tailwind指令 + 代码高亮token颜色类 + 滚动条样式 + reduced-motion适配)
7. 编写 src/types/algo.ts 第一版:
 - AlgorithmCategory 联合类型: 'linear-list' | 'stack-queue' | 'tree-binary' | 'graph' | 'sorting' | 'searching'
 - ActionType: compare | swap | move | insert | highlight | complete (6种)
 - VisualizerType: array | linked-list | tree | graph (4种)
 - VisualStep 接口: data / activeIndices / markedIndices / actionType / description / highlightLines
 - AlgorithmMeta / LogEntry / AlgorithmEntry / AlgoStore / CategoryConfig 等
 - 常量导出: CATEGORY_CONFIGS / DIFFICULTY_COLORS / SPEED_LABELS / SPEED_VALUES
8. 编写 src/context/AlgoContext.tsx 第一版:
 - 状态字段: selectedAlgorithm / categoryFilter / playState / speed / currentStep / steps / customData / useCustomData / logs
 - Action方法: selectAlgorithm / setCategoryFilter / play / pause / stepForward / stepBackward / reset / setSpeed / setCustomData / generateRandomData / loadAlgorithm / saveTestCase / loadTestCase
9. 编写 electron/main.ts: BrowserWindow 创建、M1 macOS 适配、安全策略、外部链接处理
10. 编写 src/main.tsx 和 src/App.tsx 入口文件

遇到的问题:

1. postcss.config.js 报 MODULE_TYPELESS_PACKAGE_JSON 警告 —— CJS/ESM 模块类型冲突
2. npm install electron 时二进制下载失败 (socket hang up / ETIMEDOUT)
3. tsconfig.json 的 references 引用 tsconfig.node.json 缺少 composite: true

问题分析和解决过程:

1. 将 postcss.config.js → postcss.config.mjs, tailwind.config.js → tailwind.config.mjs, 消除模块类型歧义
2. Electron 二进制从 GitHub Releases 下载, 国内网络不稳定。尝试设置 ELECTRON_MIRROR 环境变量指向 npmmirror.com 镜像, 等待网络恢复后重试成功
3. 在 tsconfig.node.json 中添加 "composite": true (后续又因与 declaration: false 冲突而移除, 改为直接删除 composite)

参与的测试或文档工作:

- 验证 TypeScript tsc --noEmit 通过
- 验证 Vite npm run build 生产构建通过 (300KB JS + 17KB CSS)
- 编写 package.json 的 electron-builder 配置 (mac arm64 dmg+zip target)

与其他成员协作的情况:

- 与成员B讨论 VisualStep 接口设计, 确认 data/activeIndices/markedIndices 字段语义
- 与成员C沟通可视化组件的 Props 接口规范
- 将类型系统同步给全体成员作为开发基准

AI 使用记录:

- 使用 Claude Code 进行项目脚手架搭建、类型接口生成、Zustand Store 实现
 - AI 生成代码约 800 行，人工修改约 20%（配置文件路径、样式细节、安全策略参数）
 - 使用 GitHub Copilot 辅助 TypeScript 类型定义的补全
-

2026年6月19日（星期四）

完成内容：

1. 扩展 types/algo.ts 第二版：
 - AlgorithmCategory 新增 'advanced'（高级专题）
 - ActionType 从 6 种扩展到 14 种：新增 delete / recurse-in / recurse-out / divide / merge / relax / visit / backtrack
 - VisualizerType 从 4 种扩展到 8 种：新增 array-dual / grid / grid+table / timeline
 - VisualStep 新增扩展字段：secondaryData / graphEdges / gridData / gridCells / dpCellUpdates / pointerPositions / pointers
 - GraphEdgeHighlight / GridCell / DPCellUpdate / PointerInfo 辅助接口
 - SpeedMultiplier 扩展到 5 档：0.25 | 0.5 | 1 | 2 | 4
2. 更新 Canvas.tsx：switch-case 分发新增 grid / grid+table 类型
3. 更新 Sidebar.tsx：categories 数组新增 'advanced'
4. 修复 generateRandomData 逻辑：从简单的"全部随机数组"改为按算法 ID 定制

遇到的问题：

1. ActionType 扩展后，LogPanel 和 ArrayVisualizer 中 ACTION_LABELS/ACTION_COLORS 报类型错误（缺少新类型）
2. GraphEdgeHighlight 的 weight 字段类型需要与 GraphVisualizer 的渲染逻辑一致
3. 部分算法（N皇后/迷宫）随机生成的数据无意义

问题分析和解决过程：

1. 在 LogPanel.tsx 的 ACTION_LABELS 中补全全部 16 种 ActionType 的中文标签和颜色
2. 在 ArrayVisualizer.tsx 的 ACTION_COLORS 中补全全部 16 种颜色映射
3. 按算法ID定制随机数据规则：N皇后生成4-8随机数、迷宫保持默认网格、图保持默认拓扑

参与的测试或文档工作：

- 验证 tsc --noEmit 零错误
- 验证 vite build 通过

与其他成员协作的情况：

- 与成员B确认新增 ActionType 是否满足全部算法需求
- 与成员D沟通 DataControl 需要适配新的输入格式

AI 使用记录：

- 使用 Claude Code 进行类型系统扩展、ActionType 枚举扩展
 - AI 提供扩展方案，人工审核类型完备性
-

2026年6月20日（星期五）

完成内容：

1. 重写 generateRandomData 函数：按 55 个算法 ID 精确匹配生成规则
 - 排序算法 → 8个随机数
 - 二分查找/插值查找 → 10个有序数 + 数组中已有的目标值
 - BST查找 → 7个BST节点 + 其中已有的目标值
 - 顺表插入 → 5元素 + 随机插入值 + 合法插入位
 - 顺表删除/链表删除 → 5元素 + 合法删除位
 - 栈/队列 → 3-6个随机值
 - N皇后 → 随机棋盘 4-8
 - 0/1背包 → 4件随机重量+4件价值+合理容量
 - LCS → 2组随机序列
 - 图算法 → 随机4-7节点+随机边+随机权重
 - 括号匹配 → 合法括号对再随机打乱
 - 中缀转后缀 → 合法数字表达式
2. 实现字符直输功能：DataControl 检测非数字字符时自动转 charCode
3. 图算法边列表输入：格式 **N, from,to,w, ...**
4. 更新 DataControl 的 isReadonly 逻辑：图算法不再置灰
5. 更新 DataControl 的输入提示：按算法类型显示不同 placeholder

遇到的问题：

1. 字符直输与数字输入模式的切换逻辑需要精确判断
2. 图算法旧格式 **[0,1,2,3,4]** 与新边列表格式需要兼容
3. 中缀转后缀随机数据生成垃圾字符（随机 ASCII）

问题分析和解决过程：

1. 针对 bracket-match 和 infix-to-postfix 开启 isSymbolMode，检测 **/[^0-9,\s\-\]/** 判断是否字符输入
2. 统一图算法采用 **[N, from,to,w, ...]** 格式，更新 defaultData
3. 中缀转后缀随机生成改为：随机数字+随机运算符交替拼接，确保格式合法

参与的测试或文档工作：

- 逐算法测试随机生成按钮，验证生成数据的合法性
- 测试字符直输功能：输入 **({[]})** 和 **3+2*4** 验证正确转换

与其他成员协作的情况：

- 与成员D协作 DataControl 改造
- 与成员B确认各算法的合法输入格式

AI 使用记录：

- 使用 Claude Code 生成 55 条随机数据规则的分支逻辑
- 使用 GitHub Copilot 辅助正则表达式编写

2026年6月21日（星期六）

完成内容：

1. GraphVisualizer 重构：支持 N 节点环形布局
 - `circularLayout(n)` 函数：N 个节点均匀分布在圆周上
 - `parseEdges(data)` 函数：解析边列表格式 `[N, from,to,w, ...]`
 - 节点标签扩展到 A-P (16个字母)
2. 更新 Dijkstra 和 BFS 的 step 生成器：解析边列表 → 构建邻接矩阵
3. 更新 GraphVisualizer 数据格式：`data = [N, dist0, dist1, ...]`
4. 更新图算法 defaultData：全部改为边列表格式 (Dijkstra/BFS/DFS/Prim/Kruskal)
5. 更新随机数据生成器：图算法产生 4-7 个随机节点 + 随机边 + 随机权重
6. 更新 DataControl 图算法输入提示和数据显示标签

遇到的问题：

1. GraphVisualizer 的边渲染使用 `graphEdges` 字段，类型为 `GraphEdgeHighlight[]` (weight 可选)，与内部 `w` 字段不一致
2. Dijkstra meta 出现重复 description 属性
3. Prim 和 Kruskal 的 defaultData 复用同一句代码，需分别定位替换

问题分析和解决过程：

1. 统一使用 GraphEdgeHighlight 的 weight 可选字段，边渲染中 `e.weight ?? 1` 兜底
2. 删除 Dijkstra meta 中重复的旧 description，保留新的边列表格式说明
3. 分别用 `generatePrimSteps` 和 `generateKruskalSteps` 作为锚点匹配替换

参与的测试或文档工作：

- 测试图自定义输入：输入 `4,0,1,5,0,2,2,1,3,1` 验证 4 节点图正确渲染
- 验证随机生成的图数据环形布局正确

与其他成员协作的情况：

- 与成员C协作 GraphVisualizer 重构
- 与成员B确认图算法步骤生成器兼容新格式

AI 使用记录：

- 使用 Claude Code 重构 GraphVisualizer，生成环形布局算法
- AI 生成代码约 200 行，人工修改约 15%

2026年6月22日（星期日）

完成内容：

1. Electron 打包配置完善：
 - 修复 electron 版本号从 `^30.5.1` → `30.5.1` (electron-builder 需要精确版本)
 - 修复 electron-builder 版本号从 `^24.13.3` → `24.13.3`
 - 修复 tsconfig.node.json 移除 composite (与 declaration: false 冲突)
 - 添加 Windows target 配置 (nsis)
2. 成功打包 Mac ARM64：产出 AlgoVisual-1.0.0-mac-arm64.dmg (92MB) + .zip (89MB)

3. 整理源代码包：AlgoVisual-source.zip (110KB, 清理 IDE 临时文件)
4. 编写 README.md：快速开始、项目结构、技术栈、功能说明

遇到的问题：

1. Windows 跨平台打包时 electron-v30.5.1-win32-x64.zip 下载超时
2. electron-builder 报 "version is a range, not a fixed version" 错误
3. npm run build 中 tsc 命令找不到 (node_modules 被清理后)

问题分析和解决过程：

1. Windows 打包网络超时 —— 建议在 Windows 环境下构建，记录在文档中
2. 将 package.json 中的 `^30.5.1` 和 `^24.13.3` 改为固定版本号
3. 重新执行 npm install 恢复 node_modules 后再构建

参与的测试或文档工作：

- 完整构建流程验证：`npm install` → `tsc --noEmit` → `vite build` → `tsc -p tsconfig.node.json` → `electron-builder --mac`
- 编写打包说明文档

与其他成员协作的情况：

- 与成员E协作打包测试
- 同步全部成员最终版本号和构建流程

AI 使用记录：

- 使用 Claude Code 解决 electron-builder 配置问题
- 查阅 electron-builder 文档确认 ARM64 打包参数

2026年6月23日（星期一）

完成内容：

1. 最终版本整理：
 - 确认 `tsc --noEmit` 零错误
 - 确认 vite build 通过 (398KB JS + 18KB CSS)
 - 确认 55 个算法全部注册
2. 修复 package.json 版本号问题后重新打包
3. 更新源代码 zip 包 (含全部文档)
4. 参与最终项目文档和开发日志的审核

参与的测试或文档工作：

- 最终全量验证：TypeScript 类型检查 + Vite 构建 + Electron 打包
- 审核 FINAL_REPORT.md 和 DEV_LOGS.md

AI 使用记录：

- 使用 Claude Code 整理文档格式

- 汇总项目数据统计

三、遇到的主要问题汇总

序号	问题描述	分析	解决方案	状态
1	CJS/ESM 模块类型冲突	postcss/tailwind 配置文件导出方式与 package.json type 字段不匹配	配置文件改为 .mjs 扩展名	✓
2	Electron 二进制下载失败	GitHub Releases 在国内网络不稳定	使用 ELECTRON_MIRROR 镜像	✓
3	TypeScript 类型检查报 20+ 错误	ActionType/VisualizerType 扩展后未同步更新所有引用	逐一补全 ACTION_LABELS/ACTION_COLORS	✓
4	随机数据对所有算法生成相同格式	未区分算法类型的输入需求	按算法 ID 定制 55 条生成规则	✓
5	electron-builder 版本范围报错	electron-builder 需要固定版本号	^30.5.1 → 30.5.1	✓
6	Windows 跨平台打包网络超时	下载 Windows Electron 二进制超时	建议 Windows 环境构建	⚠
7	node_modules 清理后 tsc 找不到	npm run build 依赖 npx 调用全局 tsc	重新 npm install	✓

四、AI 使用记录汇总

日期	AI 工具	使用场景	产出	人工修改比例
6.18	Claude Code	项目脚手架搭建	package.json/配置文件/index.html	20%
6.18	Claude Code	类型系统设计	types/algo.ts (第一版)	15%
6.18	Claude Code	Zustand 状态机	AlgoContext.tsx	20%
6.18	Claude Code	Electron 主进程	electron/main.ts	10%
6.19	Claude Code	类型系统扩展	ActionType 14种/VisualizerType 8种	15%
6.20	Claude Code	随机数据生成器	55条分支规则	25%
6.20	Claude Code	字符直输	DataControl 符号检测逻辑	20%
6.21	Claude Code	GraphVisualizer 重构	环形布局+N节点适配	15%
6.22	Claude Code	打包配置修复	electron-builder 版本号修复	10%
6.22	GitHub Copilot	TypeScript 补全	类型定义/接口实现的代码片段	30%
6.23	Claude Code	文档整理	最终项目文档/开发日志	15%

- **AI 对话总轮次**: 约 20 轮
- **AI 生成代码总行数**: 约 1200 行
- **总人工修改比例**: 约 20%

五、个人总结

通过本项目，我完整实践了从前端工程化搭建到桌面应用打包的全流程。以下是我的核心收获：

技术方面：

1. Electron + Vite + React + TypeScript 的技术栈组合在开发效率和运行性能之间取得了很好的平衡
2. Zustand 的轻量级状态管理非常适合中等复杂度的桌面应用，特别是 `getState()` 在 `setInterval` 中的使用解决了 React 状态闭包问题
3. TypeScript 严格模式下的接口先行设计有效避免了大量运行时错误，类型系统成为团队协作的"合约"
4. electron-builder 的跨平台打包需要特别注意版本号的精确匹配和网络环境

工程方面：

1. 好的项目脚手架是团队效率的基础——统一的 tsconfig、eslint、路径别名能避免大量配置碎片化问题
2. 随机数据生成器看似简单，实际需要深入理解每个算法的输入约束才能生成合法数据
3. 字符直输功能看似很小的改进，但需要同时修改 DataControl、AlgoContext 和多个算法的步骤生成器

协作方面：

1. 作为架构负责人，需要提前定义好接口规范（VisualStep/AlgoStore等）并确保团队成员理解一致
2. 及时同步类型变更（如 ActionType 扩展）避免成员B和C的代码出现类型错误
3. AI 工具在代码生成和文档编写方面效率极高，但架构决策和边界条件需要人工判断

不足与改进：

1. 图算法跨平台打包未完成，后续可在 CI/CD 环境中配置多平台构建
2. 可考虑引入离线 Electron 二进制包，避免网络依赖
3. 代码签名证书的配置可以进一步提升用户体验